Intelligent Robotics Final Project

ROS2 기반 경찰차 차단 구역 감지 및 자율 우회 주행 시뮬레이션

Felony Stop Scenario · Autonomous Detour Navigation

박세은 (2248031)

이화여자대학교 컴퓨터공학과

2026. 06. 19

프로젝트 배경: Waymo Felony Stop 사건

2025.12, LA Waymo Robotaxi 사건

경광등이 켜진 경찰차 여러 대가 교차로를 봉쇄한 felony stop 현장에 **Waymo 로보택시가 승객을 태운 채 진입**한 사건이 발생.

핵심 문제

자율주행 시스템이 **단순 객체 감지를 넘어**, 경찰차의 존재와 경광등 작동 상태를 **결합하여 현장의 상황적 맥락**을 판단해야 함.

프로젝트 목표

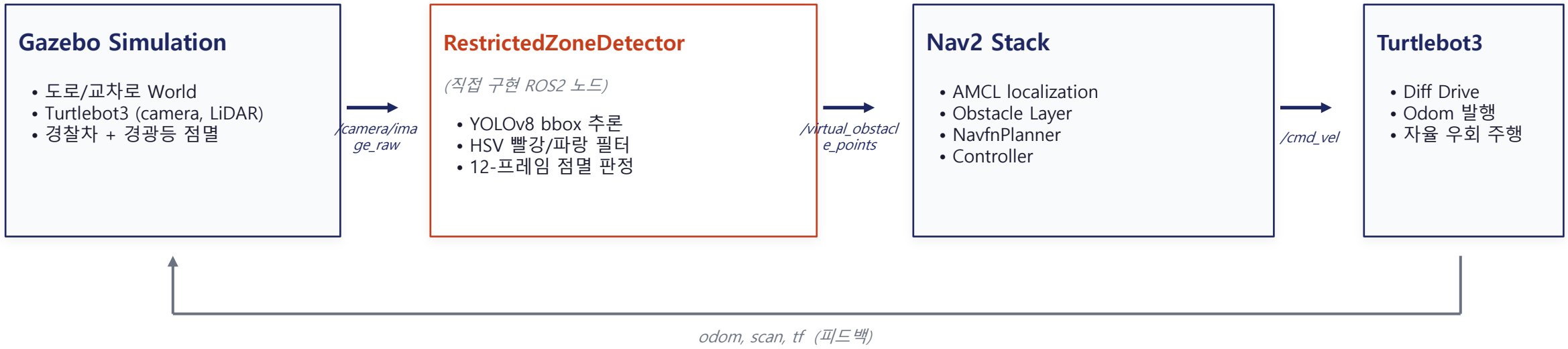
Gazebo 환경에서 경찰차 + 경광등 점멸을 기반으로 통제 구역을 판단하고, 진입 전 우회 경로를 자동 생성하는 자율주행 시스템 구현.



[그림: Waymo felony stop NBC News 캡처]

시스템 구조 (System Architecture)

ROS2 노드 및 토픽 흐름



ROS2 패키지 구조: felony_stop_sim

```
felony_stop_sim/
├── worlds/intersection.world          # 도로 교차로 SDF 월드
├── models/police_car/                  # 경찰차 모델 + 경광등 FlashLightPlugin
├── maps/intersection_map.yaml          # SLAM으로 생성한 occupancy grid
├── launch/intersection_world.launch.py
├── launch/restricted_zone_detector.launch.py
└── nodes/restricted_zone_detector.py   # 통제 구역 판단 노드 (직접 구현)
```

구현 방법 ①: 시뮬레이션 환경 구축

도로 교차로 World

SDF 1.6으로 직접 작성. Waymo 사건 교차로를 단순화하여 4m 폭 십자형 도로와 모서리 4개의 건물 블록 구성.

Turtlebot3 Waffle Pi

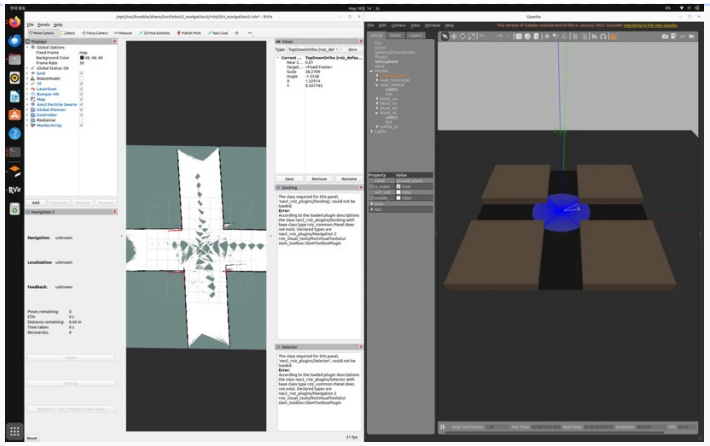
차량 역할. RGB 카메라(/camera/image_raw)와 360° LiDAR(/scan) 보유. (0, -7)에서 spawn하여 북쪽으로 자율주행.

SLAM 맵 생성

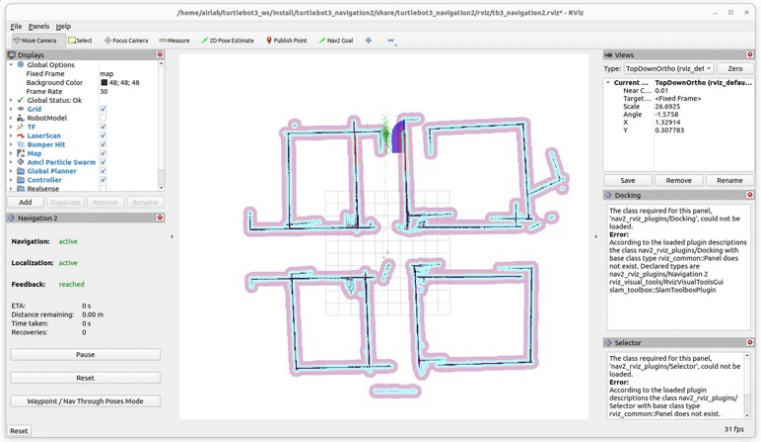
slam_toolbox로 환경 주행하며 occupancy grid 생성(5cm/픽셀). nav2_map_server로 .pgm + .yaml 저장.

Nav2 자율주행 검증

AMCL + NavfnPlanner. 남→북 자율 주행 검증 완료 (Recoveries 0회).

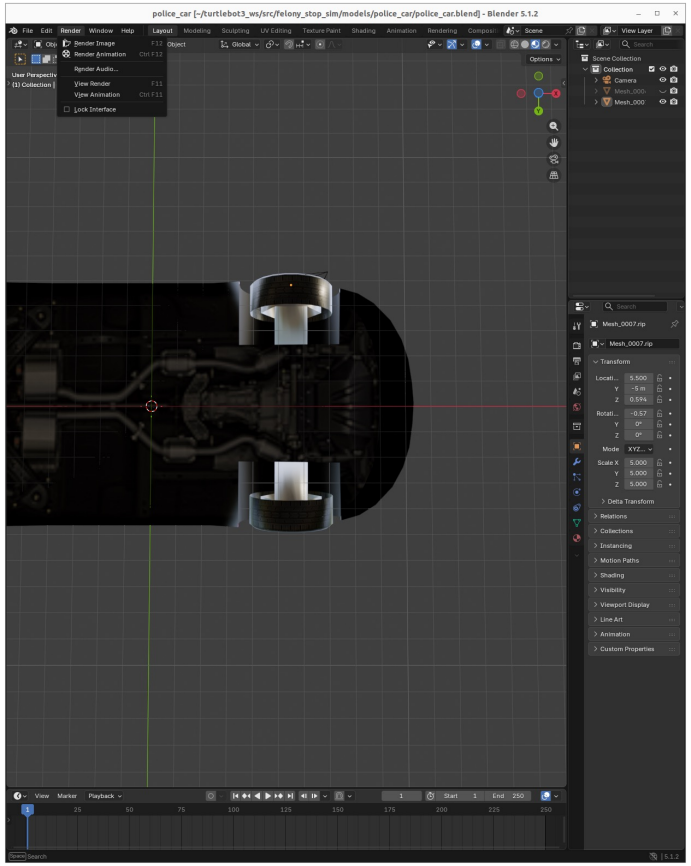


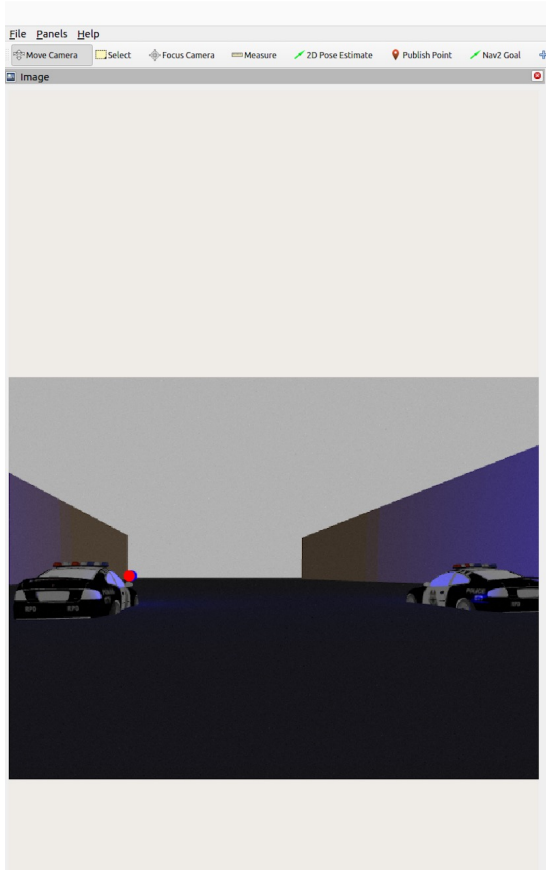
[그림 4-1: Gazebo 도로 교차로 환경]



[그림 4-2: RViz Nav2 자율주행 (Goal reached)]

구현 방법 ②: 경찰차 모델 및 경광등 점멸





[그림 5-1: 카메라에 잡힌 경찰차 + 경광등 점멸 (POLICE/RPD 텍스처)]

모델 변환 파이프라인

Sketchfab (CC BY) → Blender → .obj → SDF
mesh scale 0.02 (차 길이 ~50cm, Turtlebot3 ~30cm 대비 적정 비율)
static=true로 정적 장애물 처리

FlashLightPlugin 경광등

차 지붕에 빨간/파란 **point light + emissive sphere** 배치.
duration 0.5s, interval 0.5s 교대 점멸.
카메라 영상에서도 색상 식별 가능하도록 emissive material 적용.

교차로 배치

교차로 북쪽 차선에 경찰차 2대를 동서 방향으로 나란히 배치. Turtlebot3가 남→북 진행 시 정면 차단.

구현 방법 ③: RestrictedZoneDetector 노드

핵심 ROS2 노드 — 카메라 영상에서 통제 구역을 판단하고 가상 장애물을 publish

① 경찰차 감지

YOLOv8n fine-tuned 모델로 bounding box 추론.

- Roboflow police_car 데이터셋
- Gazebo 합성 이미지 혼합

② HSV 색상 필터

bbox ROI에서 빨강/파랑 픽셀 비율.

- 빨강 H: 0-10, 170-180
- 파랑 H: 100-130
- S, V: 100-255

③ 다중 프레임 판정

12-프레임 윈도우 누적 비율.

- 단일 프레임 깜빡임 회피
- 임계값 0.30 초과시 ON
- 빨강+파랑 동시 만족

④ 가상 장애물 publish

PointCloud2 메시지로 변환.

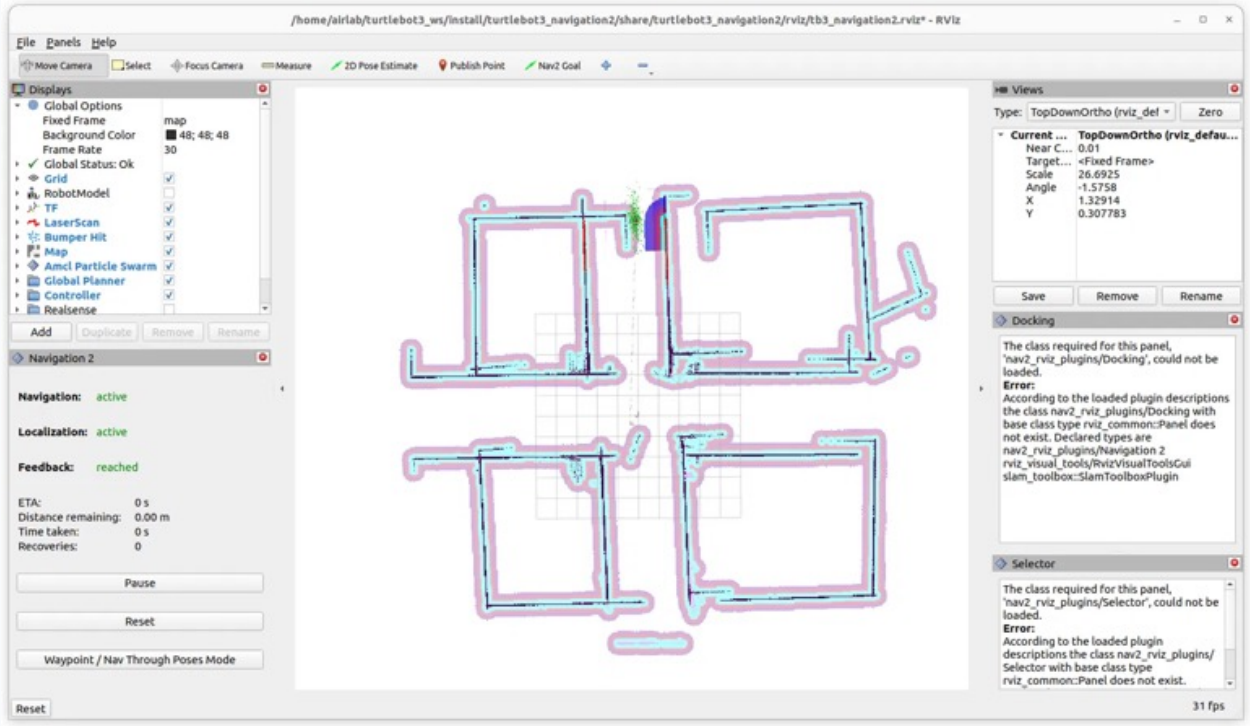
- /virtual_obstacle_points
- Nav2 obstacle_layer 구독
- Costmap 동적 업데이트

핵심 로직 (restricted_zone_detector.py)

```
# HSV 빨강/파랑 마스크 (제안서 명시 임계값)
red_mask  = cv2.inRange(hsv, [0,100,100], [10,255,255]) | cv2.inRange(hsv, [170,100,100], [180,255,255])
blue_mask = cv2.inRange(hsv, [100,100,100], [130,255,255])

# 12-프레임 누적 윈도우 기반 점멸 판정
self.red_history.append(red_ratio > 0.02)        # deque(maxlen=12)
light_active = (sum(red_history)/12 > 0.30) and (sum(blue_history)/12 > 0.30)
```

결과 ①: Nav2 자율주행 기본 동작 검증



[그림 7-1: 장애물 없는 환경에서 남→북 자율주행 (RViz)]

시나리오

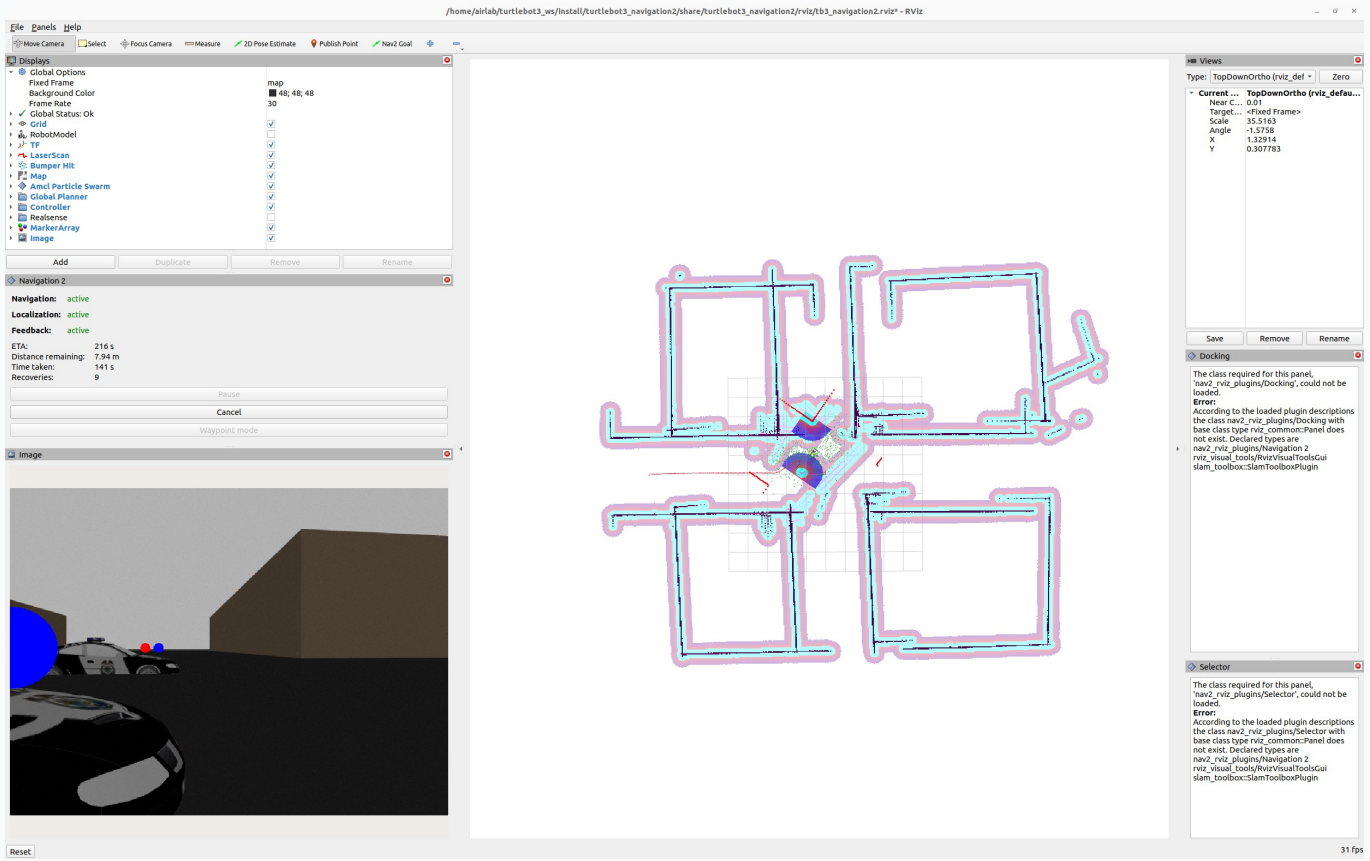
도로 남쪽 끝 (0, -7) → 북쪽 끝까지
장애물 없는 환경에서 자율 주행

✓ Success

Goal reached

Recoveries	**0**
Distance left	**0.00 m**
ETA	**0 s**
Localization	**active**

결과 ②: 경찰차 차단 시나리오 시연



[그림 8-1: 경찰차 두 대가 교차로를 차단 (카메라 시점)]

시나리오

경찰차 2대가 북쪽 차선 차단.
경광등 ON 상태로 점멸 중.

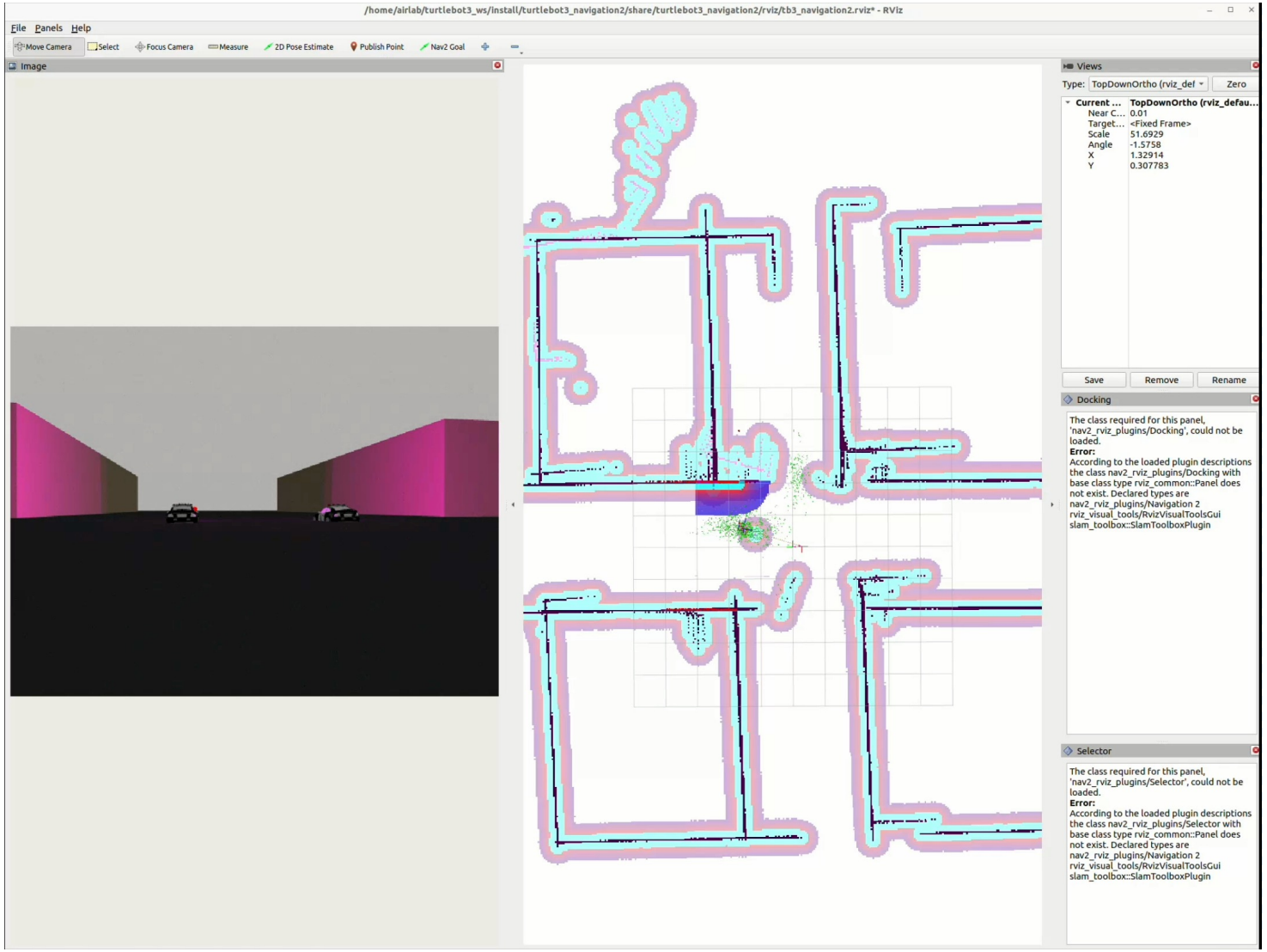
관찰 결과

✓ 경찰차가 **Nav2 costmap에 장애물로 인식** (LiDAR 기반)

✓ 카메라 영상에 **POLICE/RPD 텍스처 및 경광등 점멸** 식별 가능

✓ 정적 장애물로서 자율주행 차단 발동 — **Recovery 행동 후 경로 실패** 확인

결과 ②: 경찰차 차단 시나리오 시연



시나리오

경찰차 2대가 북쪽 차선 차단.
경광등 ON 상태로 점멸 중.

관찰 결과

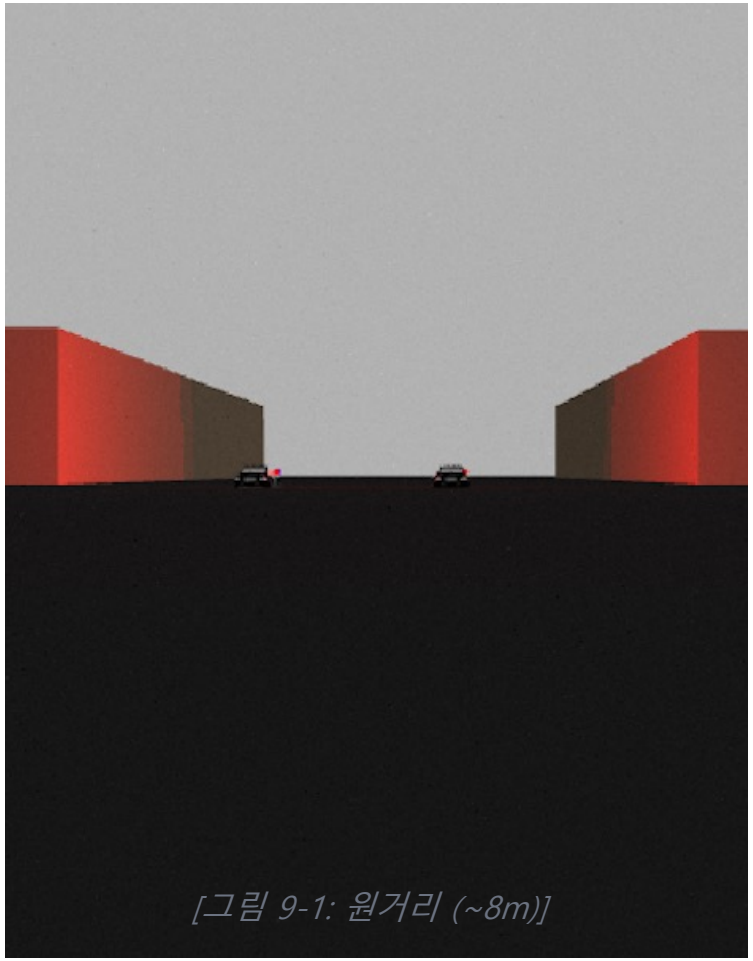
✓ 경찰차가 **Nav2 costmap에 장애물로 인식** (LiDAR 기반)

✓ 카메라 영상에 **POLICE/RPD 텍스처 및 경광등 점멸** 식별 가능

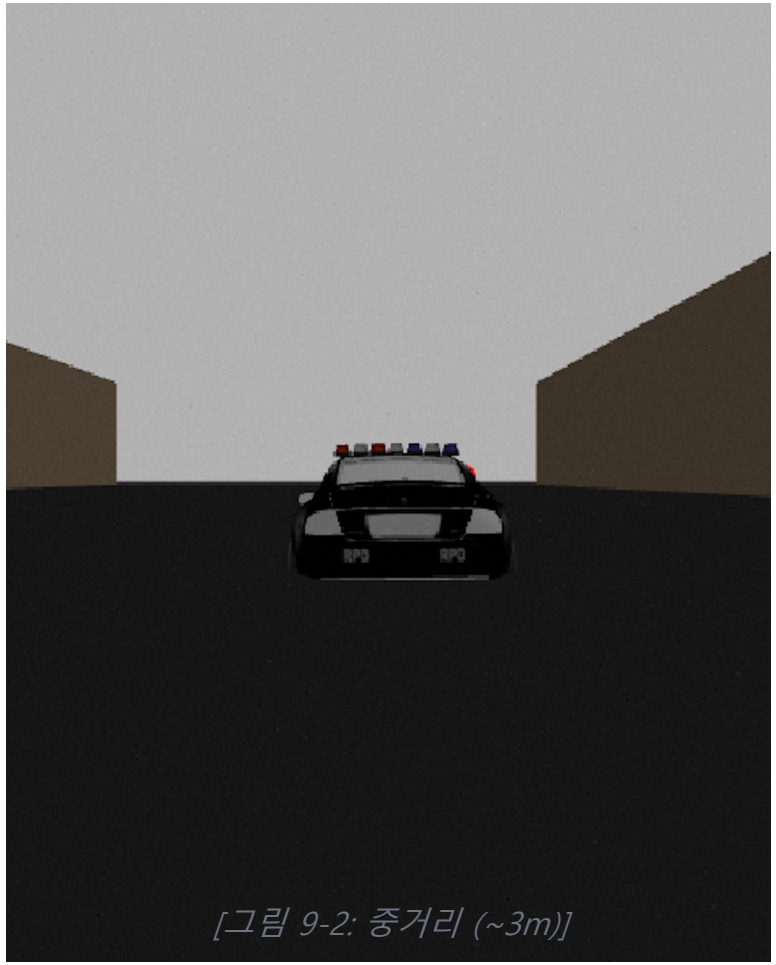
✓ 정적 장애물로서 자율주행 차단 발동 — **Recovery 행동 후 경로 실패** 확인

결과 ③: 경광등 점멸의 카메라 인식

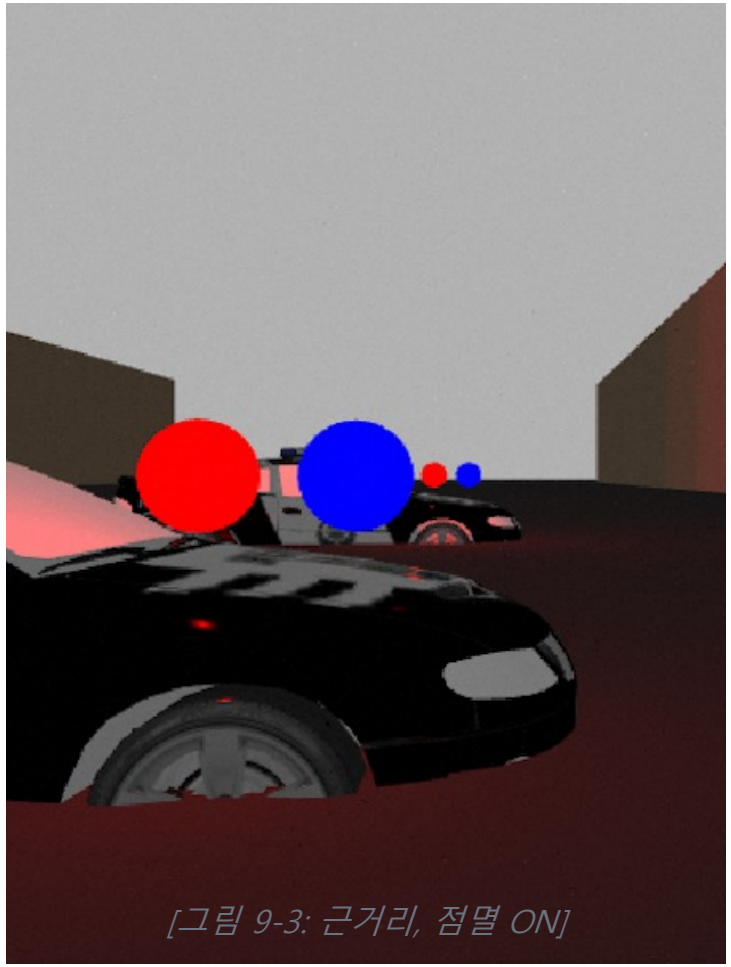
YOLO 학습 입력으로 사용할 합성 이미지의 예시 — 거리/각도별 다양성 확보



[그림 9-1: 원거리 (~8m)]



[그림 9-2: 중거리 (~3m)]



[그림 9-3: 근거리, 점멸 ON]

관찰: 거리/각도별로 경찰차 외형과 경광등 색상이 카메라에 안정적으로 캡처됨. sim-to-real domain gap 보완용 합성 이미지로 활용 예정.

어려웠던 점 ①: RTX 50 + Gazebo OGRE 호환성

문제

RTX 5070 Laptop GPU에서 **gzclient(Gazebo GUI) 실행 시 assertion 실패**:
px != 0 in gazebo::rendering::Camera
Turtlebot3 모델/Burger 무관, 모든 GUI 실행에서 재현됨.

진단

Gazebo Classic 11이 사용하는 OGRE 1.9 렌더링 엔진은 2025년 1월 EOL.
RTX 50 시리즈의 새 NVIDIA 드라이버(595)와 호환되지 않음.

시도한 해결책

PRIME 프로파일 전환, LIBGL 환경변수, GAZEBO_MODEL_DATABASE_URI 차단, ~/.gazebo 캐시 초기화 — 일시적으로만 동작

우회 전략

Headless Gazebo + RViz 시각화

- gzserver는 정상 동작 → 시뮬레이션 백엔드 + 토픽 발행 영향 없음
- gzclient만 비활성화 (launch에서 제외)
- RViz로 LaserScan, Image, Map, 로봇 위치 모두 시각화
- 카메라 영상은 /camera/image_raw 토픽으로 직접 확인

학습한 점

프로젝트 핵심은 GUI 비주얼이 아니라 토픽 기반 자율주행 알고리즘. 도구 제약이 본질을 막지 않도록 설계 분리 필요성 체감.

어려웠던 점 ②: AMCL 위치추정 ↔ 맵 origin 정렬

문제

RViz에서 2D Pose Estimate으로 거북이 위치 지정 → AMCL은 **잘못된 map → odom 변환**을 발행. Gazebo 실제 거북이는 도로에 있으나 RViz 시각화는 엉뚱한 위치 + 반대 방향 주행 발생.

원인

SLAM이 자동 산출한 맵 **origin (-30.9, -24.8)**이 시뮬레이션 월드 좌표계와 일치하지 않음. 2D Pose Estimate은 지도 픽셀 좌표 기준이라 클릭한 위치가 실제 월드 좌표와 다르게 변환됨.

해결: /initialpose 직접 발행

RViz 클릭 대신 ROS2 토픽으로 정확한 월드 좌표 강제 지정:

```
ros2 topic pub --once /initialpose \
  geometry_msgs/msg/PoseWithCovarianceStamped \
  '{header: {frame_id: map},
    pose: {pose: {
      position: {x: 0.0, y: -7.0, z: 0.0},
      orientation: {w: 1.0}}}}'
```

학습한 점

GUI 클릭은 사용자 친화적이지만 좌표 정밀도가 떨어짐. 시뮬레이션 환경에서 정확한 spawn 좌표를 알면 토픽으로 직접 지정하는 게 디버깅에 효율적.

개선 방향 및 향후 작업

현재 RestrictedZoneDetector는 HSV 색상 필터와 가상 장애물 publish 구조까지 완성. YOLO 추론과 Nav2 동적 통합이 다음 단계.

1

YOLOv8 Fine-tuning

Roboflow police_car 데이터셋(1,200장) + Gazebo 합성 이미지 200-300장 혼합 학습. sim-to-real domain gap 보완. mAP 측정.

2

Nav2 Obstacle Layer 통합

/virtual_obstacle_points → Nav2 costmap obstacle_layer.observation_sources에 등록. 감지 시 실시간 우회 경로 재계산 검증.

3

3가지 시나리오 비교 시연

(1) 통제 구역 없음 (2) 경광등 OFF: 우회 없이 통과 (3) 경광등 ON: 우회. 시나리오 2의 false positive 방지가 핵심 검증 포인트.

4

정량 평가

mAP (감지 성능), 목표 도달 성공률, 통제 구역 진입 여부, 우회 경로 생성 시간 및 경로 길이를 시나리오별로 비교 기록.

프로젝트 요약

완성된 결과물

- 도로 교차로 Gazebo World (SDF 직접 작성)
- Turtlebot3 + Nav2 자율주행 환경 구축
- SLAM 기반 occupancy grid 맵 생성/저장
- 경찰차 3D 모델 통합 (Blender 변환)
- 경광등 점멸 (FlashLightPlugin, 빨강/파랑 교대)
- RestrictedZoneDetector ROS2 노드 구현 (HSV + 다중 프레임 판정 + 가상 장애물 publish)
- RTX 50 + AMCL 트러블슈팅 및 우회 전략 확립

기여 포인트

1. 단순 객체 감지가 아닌 **상황적 맥락 판단**—경찰차 존재와 경광등 작동 상태를 결합.

2. **다중 프레임 윈도우** 도입으로 점멸 신호의 단일 프레임 깜빡임을 안정적으로 처리.

3. **PointCloud2 가상 장애물** 방식으로 Nav2 obstacle layer 플러그인을 직접 작성하지 않고도 동적 장애물 반영.

4. **Sim-to-real domain gap** 인식 — Gazebo 합성 이미지와 실제 데이터셋 혼합 학습 계획.

Thank you

박세은 (2248031) · Intelligent Robotics Final Project